# ATLAS-DEP-001

Deployment Guide — Version 2

| | |
|---|---|
| **Project** | ATLAS Master Flightline Clock |
| **Document ID** | ATLAS-DEP-001 |
| **Version** | 2.0 |
| **System Version** | v1.6.0 |
| **Date** | July 2026 |
| **Status** | CURRENT AUTHORITATIVE — supersedes v1 |

## 1. Purpose

This guide defines the deployment philosophy, current deployment method, and deployment expectations for the ATLAS Master Flightline Clock. It covers the current macOS development deployment and establishes the baseline for the forthcoming Windows field deployment process.

## 2. Deployment Philosophy

ATLAS is intended to function as a dedicated operational appliance. The operator should interact with ATLAS, not with the host operating system, except during maintenance or administrative tasks.

This principle applies whether ATLAS is deployed on macOS or Windows.

> The system should start predictably, run reliably, and recover cleanly. The operator should never need to touch a terminal, file manager, or system settings during normal competition operation.

## 3. Runtime Dependencies

| Dependency | Version | Notes |
|---|---|---|
| Python | 3.12 | 3.14 caused Qt platform plugin issues — use 3.12 |
| PySide6 | 6.11.1 | Qt for Python GUI framework |
| Pydantic | 2.x | Data model validation |
| SQLite | Built-in | No separate installation required |

| | | |
|---|---|---|
| All other packages | See requirements.txt | Install via pip into virtual environment |

> Do not use Python 3.14. PySide6 6.11.1 has known Qt platform plugin compatibility issues with Python 3.14 on macOS. Python 3.12 is the validated runtime.

## 4. Current macOS Development Deployment

### 4.1 Project Structure

The project lives at `~/Project_Atlas/` on the development Mac. The canonical source layout is:

```
Project_Atlas/
  src/
    main.py
    core/
      engine.py
      database.py
      models.py
    gui/
      windows.py
      led_simulator.py
      lcd_display.py
      fonts.py
      themes.py
  data/
    atlas.db
  .venv/
  requirements.txt
  run_atlas.sh
  atlas.sh
```

### 4.2 Launch Method

From the project root directory:

```
source .venv/bin/activate
export PYTHONPATH=$PYTHONPATH:$(pwd)/src
python src/main.py
```

Or using the full launcher script which also handles Qt platform plugin pathing:

```
./run_atlas.sh
```

### 4.3 What run_atlas.sh Does

- Changes to the project directory
- Activates the virtual environment
- Sets `QT_QPA_PLATFORM_PLUGIN_PATH` for macOS Qt compatibility
- Sets `QT_QPA_PLATFORM=cocoa`
- Sets `PYTHONPATH` to include `src/`
- Launches `python src/main.py`

## 5. Startup Modes

| Mode | Setting | Behaviour | Use Case |
|---|---|---|---|
| Normal (default) | `auto_boot_enabled = "0"` | Launches operator console and public display | Development and active schedule management |
| Auto-Boot / Appliance | `auto_boot_enabled = "1"` | Launches public display only | Field deployment — display-only operation |

> Auto-boot mode defaults to OFF. This is intentional — the system is in active development and should not lock into appliance mode unexpectedly. Enable only when deploying to a dedicated field machine.

## 6. Display Backend Selection

| Backend | Setting Value | Best For |
|---|---|---|
| LED Simulator | `simulator` | Development, testing, small displays |
| LCD Monitor | `lcd` | Field deployment on standard monitors or TVs |

Backend selection is made in the Settings tab and takes effect after restarting ATLAS.

## 7. Settings That Require Restart

- Display backend (LED Simulator vs LCD Monitor)
- Display size (128x64 vs 64x64)
- Auto-boot toggle

### Settings That Apply Immediately

- Brightness slider (live update via Qt signal)

## 8. Known macOS Issue — Qt Platform Plugin

If ATLAS fails to start with a Qt platform plugin error:

```
qt.qpa.plugin: Could not find the Qt platform plugin "cocoa"
```

Resolution:

```
cd ~/Project_Atlas
source .venv/bin/activate
pip install --force-reinstall PySide6
```

If the plugin exists but still fails, set the path explicitly:

```
export QT_QPA_PLATFORM_PLUGIN_PATH=$(python -c \
  "import PySide6, os; print(os.path.join(os.path.dirname(PySide6.__file__), \
  'Qt', 'plugins'))")
python src/main.py
```

## 9. Appliance Configuration Expectations

For field deployment, the host system should be configured to:

- Disable sleep and screen savers during operation
- Suppress unnecessary system notifications
- Minimise automatic update interruptions
- Launch ATLAS predictably on startup
- Restore operation cleanly after power cycle where practical

## 10. Cross-Platform Position

ATLAS business logic and core GUI stack are cross-platform in principle. The following areas are expected to differ between macOS and Windows deployments:

| Area | macOS | Windows |
|---|---|---|

| | | |
|---|---|---|
| Startup scripts | .sh bash scripts | .bat or .ps1 scripts — to be created |
| Auto-launch on boot | LaunchAgent plist | Task Scheduler or Startup folder |
| Qt platform plugin | cocoa | windows (automatic) |
| PYTHONPATH setting | export in shell | set in batch file or environment |
| UI wording | "EXIT TO MACOS" | Should be generalised |
| Sleep/notification control | System Preferences | Power Options / Group Policy |

## 11. Known Deployment Risks to Address

| Risk | Priority | Action Required |
|---|---|---|
| Database path uses relative `data/atlas.db` | HIGH | Harden to project-relative absolute path before Windows deployment |
| PYTHONPATH must be set at launch | HIGH | Encapsulate in launcher script for all platforms |
| macOS-specific Qt platform env vars in run_atlas.sh | MEDIUM | Create separate Windows launcher |
| UI text "EXIT TO MACOS" | LOW | Generalise to "EXIT TO DESKTOP" or platform-detect |
| Legacy modules in-tree | LOW | Document as legacy or formally retire |

## 12. Immediate Deployment Priorities

1. Harden database path handling
2. Define and test Windows deployment process
3. Create Windows launcher script
4. Define field installation checklist
5. Define post-deployment verification procedure
6. Validate appliance-mode startup on target deployment hardware